

NGÔ CÔNG BẰNG

FRONT-END AND BACK-END DEVELOPER

Hi, my name is Bang. I'm a software engineer with over 10 years of experience in designing, developing, and maintaining full-stack applications. I have a strong background in backend development using technologies like .NET/C#, RESTful APIs, and SQL Server, as well as solid frontend experience with HTML, CSS, JavaScript, and frameworks such as Vue.js or React. I'm also skilled in working with databases — including writing complex queries, optimizing performance, and handling data modeling. Over the years, I've been involved in building scalable enterprise systems, collaborating across teams, and ensuring code quality through clean architecture and best practices. I enjoy learning new technologies and continuously improving both my technical and soft skills. Outside of work, I like exploring nature, working on home projects, and staying updated with tech trends

1. EDUCATION

Dalat University Information Technology

2. SKILLS

1. Frameworks & Technologies:

Angular, ASP.NET Core Web API, C#,
Python, EF Core, ReactJs, VueJs

2. Database:

Mysql, SQL Server, Oracle, Postgresql

3. Cloud:

Azure Devops

4. Source Control:

Git, SVN

3. LANGUAGE

English Limited working proficiency (255 - 600 TOEIC)

4. WORKING EXPERIENCE

Time	Company
7/2020 - 7/2025	LTS GROUP
12/2017 - 7/2020	Thiên Hoàng
1/2011 - 12/2017	BuCA

5. SOME PROJECTS

1. Accounting Software Investor(1/2016 - 9/2016)

Position: Project Manager

Team size: 7

Project description:

Project Overview:

The Investor Accounting System is a specialized financial management platform tailored for organizations managing investment capital. It adheres to regulatory accounting standards and supports core financial operations such as voucher management, asset tracking, payroll, and financial reporting.

The system provides a centralized environment for recording, tracking, and analyzing all financial transactions related to investment projects, with robust user access control and audit capabilities.

Main Functional Modules:

1) System User Management

Manage user accounts and role-based permissions.

Support authentication, activity logging, and administrative control for security and compliance.

2) Common Items Management

Define and maintain master data, including:

Chart of accounts

Cost centers and projects

Partner and supplier information

Payment methods, tax types, banks

3) Voucher Management

Covers the full lifecycle of financial vouchers across multiple categories:

Cash Vouchers: Cash receipts and payments

Deposit Vouchers: Bank transfers and deposits

Salary Vouchers: Payroll calculation and disbursement

Asset Vouchers: Fixed asset acquisition, depreciation, and movement

Item Vouchers: Material and inventory management

Each voucher type supports input, validation, approval workflow, and posting to the general ledger.

4) Reporting

Generate regulatory and internal reports, including:

General ledger, trial balance, and journal entries

Cash flow and bank reconciliation

Payroll reports and tax declarations

Depreciation and fixed asset reports

Reports can be exported in Excel or PDF format and scheduled monthly or quarterly.

Responsibilities:

Check deadlines for developers to ensure tasks are on track and delivered on time.

Join in solving complex coding tasks, especially those that require high-level logic or architecture decisions.

Support and assist team members in resolving issues, reviewing code, and improving overall productivity.

Technologies:

Visual 2012 C#, SQL Server 2008R2, Devexpress, ReportSharpShooter, TFS, Resharper...

2. Women's Union VN(1/2021 - 7/2022)

Position: Fullstack

Team size: 10

Project description:

Project Overview:

Manages user accounts, roles (admin, approver, member, etc.), and access permissions.

Supports secure login, password management, and role-based authorization by department or unit.

Main Functional Modules:

1) System User Management

Manages user accounts, roles (admin, approver, member, etc.), and access permissions.

Supports secure login, password management, and role-based authorization by department or unit.

2) Profile Member

Maintains complete records of union members, including:

Personal information (name, date of birth, gender, department, position)

Employment history

Awards and disciplinary actions

Party/union affiliation, education level, certifications

Allows searching, updating, and tracking changes in member profiles.

3) Rated Emulation (Merit Evaluation)

Supports the periodic evaluation process (monthly, quarterly, yearly) for individuals and teams.

Calculates performance scores, records achievements, and recommends rewards.

Covers recognition titles such as “Outstanding Employee” or “Emulation Flag.”

Includes multi-level review and approval workflows.

4) Reporting

Generates detailed reports and statistics on union members based on age, gender, position, etc.

Provides summaries of merit results across periods.

Allows exporting reports in Word, Excel, or PDF formats for meetings and performance reviews

Responsibilities:

Implemented both backend and frontend functionalities of the project.

Technologies:

Visual 2019 C#, SQL Server 16, Web API, Angular 6, Jira

3. Bizbone(7/2022 - 12/2023)

Position: Full Stack

Team size: 10

Project description:

Project Overview:

This project is an enterprise-wide management system designed to support and streamline operations across multiple functional departments. It includes the following core modules:.

Main Functional Modules:

1) Accounting module

Manages financial transactions, budgeting, expense tracking, invoices, and financial reporting. Integrates with other modules to ensure realtime financial visibility.

2) Human Resources (HR) Module

Handles employee records, recruitment, performance evaluations, leave management, and payroll integration.

3) Contract Management Module

Supports the lifecycle of contracts, including creation, versioning, approval workflows, renewals, and compliance tracking.

4) Program/Project Management Module

Oversees program execution, timelines, resource allocation, task assignments, and progress tracking across teams.

5) Asset Management Module

Manages the lifecycle of organizational assets, including acquisition, depreciation, maintenance scheduling, and asset transfer between departments or locations.

6) IT Service Management Module

Supports help desk functionality, incident tracking, hardware/software requests, and IT workflow automation.

7) Business Operations Module

Facilitates business process management, KPI monitoring, and reporting dashboards for strategic decision-making

Responsibilities:

Responsible for the Business, Contract, and Program modules, handling both backend and frontend development.

Developed core business logic, APIs, and database schemas for each module using C#, Entity Framework Core (Code-First), and SQL Server.

Built and maintained frontend components with Vue.js 2, ensuring a responsive and user-friendly interface.

Designed workflows and integrated functionalities across modules to support end-to-end processes, such as program planning, contract lifecycle management, and business approval flows.

Collaborated with cross-functional teams to align business requirements with technical implementation

Technologies:

Entity Framework Core (Code-First) Vue.js 2 C# (.NET Core/.NET Framework) SQL Server OData Azure Git.

4. WorkforceHQ(1/2024 - 11/2024)

Position: Backend Developer

Team size: 9

Project description:**Project Overview:**

A Human Resource Management system encompassing

Main Functional Modules:**1) Employee Information**

Personal and contact details, organizational roles, and employment history.

2) Performance Management

Goal setting, performance reviews, feedback cycles, and appraisal tracking

3) Certification Tracking

Covers the full lifecycle of financial vouchers across multiple categories:

Cash Vouchers: Cash receipts and payments

Deposit Vouchers: Bank transfers and deposits

Salary Vouchers: Payroll calculation and disbursement

Asset Vouchers: Fixed asset acquisition, depreciation, and movement

Item Vouchers: Material and inventory management

Each voucher type supports input, validation, approval workflow, and posting to the general ledger.

4) Document Management

Secure storage and versioning of HR documents such as contracts, policies, and employee records. This platform centralizes all HR processes to streamline data access, automate review workflows, and ensure compliance across the organization.

Responsibilities:

Implement and maintain the Certification Management module, including certificate issuance, renewal tracking, and compliance workflows.

Develop and support the Document Management subsystem for secure storage, version control, and retrieval of HR documentation.

Design and build notification services that: Send periodic reminders (e.g., upcoming certificate expirations) to employees. Trigger real-time alerts whenever a new certificate is issued or approved.

Ensure reliable scheduling and delivery of notifications via the Azure Function App and SQL Server backed job tables.

Collaborate with frontend developers (Vue 3) to integrate notification UIs and document/certification dashboards

Technologies:

- SQL Server as the primary relational database
- Azure Function App for notification processing
- C# for backend logic and data access- Vue 3 for building the frontend user interface
- Entity Framework Core (Code-First) for data modeling and migrations

5. WorkMax(12/2024 - 1/2025)

Position: Backend Developer

Team size: 4

Project description:

Project Overview:

This project focuses on managing employee time tracking based on locations and devices.

The system allows employees to check in/out by task, capturing exact working time per task. Managers are provided with interfaces to review and approve timesheets, and track the actual time spent on each task.

It also includes budget tracking features, where budgets are assigned to each location and task, and compared against actual completion time.

Additionally, the system supports overtime planning and monitoring, helping management make informed decisions based on time usage, deviations, and over-budget trends.

Responsibilities:

Accepts the following parameters from the client over a real-time hub connection: fromDate and toDate (DateTime) defining the period to inspect

Queries the database for all unapproved time entries the specified user has logged on that task between fromDate and toDate.

Aggregates the total declared hours and compares them against the task's preconfigured time budget for that period.

Determines whether the user has exceeded their allotted hours (i.e., accrued overtime) and calculates the exact overtime amount.

Returns a structured response back to the calling client via SignalR, containing: totalDeclaredHours isOvertime (bool) overtimeHours (double)

This real-time API allows front-end applications and managers to immediately see whether a user's logged time over a specific date range will exceed their budgeted hours—enabling proactive adjustment before formal approval.

Technologies:

In this project, I implemented backend logic using C# and PostgreSQL, developed RESTful APIs, and integrated real-time features with SignalR. On the frontend, I worked with Angular and ReactJS to deliver a responsive UI.

6. SafetyHQ Data Migration Function App(2/2025 - 7/2025)

Position: Implementing features on an Azure Function App

Team size: 7

Project description:

This project involved migrating all critical data from the legacy application into our brand-new platform by leveraging a serverless architecture. Key components included:

Azure Function App

Orchestrated the end-to-end migration pipeline Executed discrete “migration functions” for each data domain

GraphQL Integration

Queried legacy data endpoints (tables, forms, users, lessons, etc.) via GraphQL Transformed and validated the payload before ingestion

Azure Table Storage

Persisted migrated records in a highly scalable NoSQL store Maintained partition keys by data domain (e.g. “Users”, “Forms”, “Lessons”) for efficient lookups

Core Responsibilities:

Extract: Built GraphQL queries to pull complete datasets—user profiles, form definitions, course lessons, and relational tables.

Transform: Normalized legacy schemas to fit the new data model, including field mappings, data cleansing, and enrichment logic.

Load: Employed Azure Function App triggers (HTTP and Timer) to batch-load transformed records into Azure Table Storage, ensuring idempotency and retry policies.

Validation & Reporting: Implemented post-migration checks and generated summary reports for record counts, error rates, and audit logs. This serverless migration solution enabled a fully automated, scalable data transfer that minimized downtime and ensured data integrity across millions of legacy records

Responsibilities:

Form Data Includes: Form definitions and metadata Field values and input types Validation rules and conditional logic User-submitted entries (responses) Attached files and media Timestamps and audit information

Technologies:

Azure Function App: Serverless compute for hosting and executing business logic.- Azure Table Storage: Storing migrated and processed data in a scalable, NoSQL table format.- GraphQL Queries: Retrieving and transforming data from the legacy system via GraphQL endpoints.

**7. My Project MicroservicesSolution (2026)(
<https://github.com/bangnc/LearnMicroservice>)**

Position: Backend Developer

Team size: 1

Project description:

Project Overview:

MicroservicesSolution is a backend system designed using Microservices Architecture combined with Clean Architecture principles.

The system is composed of multiple independent services such as API Gateway, AuthService, UserService, and OrderService. Each service is responsible for a specific domain and can be developed, deployed, and scaled independently.

An API Gateway is used as a single entry point to route client requests to appropriate services. The project also integrates Docker for containerized deployment, ensuring consistency across environments.

The goal of the project is to build a scalable, maintainable, and modular backend system that follows modern software architecture practices.

Responsibilities:

Designed and implemented a Microservices-based architecture from scratch.

Built an API Gateway to handle routing and communication between client and services.

Developed multiple independent services:

Authentication Service (user authentication & authorization)

User Service (user management)

Order Service (order processing)

Applied Clean Architecture in each service, separating:

Domain

Application

Infrastructure

API layers

Implemented RESTful APIs for communication between services.

Structured reusable components using shared BuildingBlocks.

Configured Docker and docker-compose to run and manage multiple services simultaneously.

Ensured loose coupling and high cohesion between services.

Designed the system to be scalable and maintainable for future extensions..

Technologies:

Backend: C#, .NET

Architecture: Microservices, Clean Architecture

Containerization: Docker, Docker Compose

API: RESTful APIs

8. My Project Frontend (2026)(<https://github.com/bangnc/https://github.com/bangnc/FE>)

Position: Frontend Developer

Team size: 1

Project description:

Project Overview:

Developed a modern frontend application using Vue 3, TypeScript, and Vite, designed with a Feature-based Architecture that aligns with backend Microservices structure.

The project is organized by business domains (Auth, User, Order, Dashboard), where each feature is self-contained and independently maintainable. This approach improves scalability, code separation, and long-term maintainability.

The system communicates with multiple backend microservices through an API Gateway and handles authentication, state management, and UI logic

in a modular way.

Responsibilities:

Designed and implemented a scalable frontend architecture using Feature-based structure (domain-driven approach).

Built reusable and modular UI components following a shared component system.

Integrated RESTful APIs via Axios with centralized API handling layer.

Implemented authentication flow using JWT (login, token decode, protected routes).

Managed global and feature-level state using Pinia + persisted state plugin.

Structured application into independent features:

Auth (login/register/OTP handling)

User management

Order processing

Dashboard analytics

Configured Vue Router with lazy-loading routes per feature module.

Applied form validation using VeeValidate + Zod schema validation.

Enhanced UX with loading states, notifications, and route guards.

Ensured maintainable codebase with separation of concerns (core / shared / features).

Technologies:

Framework: Vue 3, Vite, TypeScript

State Management: Pinia, pinia-plugin-persistedstate

Routing: Vue Router 4 (lazy loading)

UI/UX: TailwindCSS, Vue Toastification, NProgress, Lucide Icons

Form & Validation: VeeValidate, Zod

HTTP Client: Axios

Utilities: Dayjs, UUID, QS, VueUse

Code Quality: ESLint, Prettier, Husky, Commitlint

6. ACTIVITY

Fiction movies, music and Team Building

7. CONTACT

Email: bangnc85@gmail.com

Phone: 0989135468